CORTEXIS SOLUTION HUB

Production Architecture & Deployment Blueprint: Hostinger KVM 4 + Cloudflare (CDN/WAF/SSL) + Nginx + Docker

Stack: React Frontend + Node.js Backend + PostgreSQL 16 + Redis 7	**Infrastructure:** Hostinger KVM 4 (Ubuntu 24.04 LTS)	**Security & Edge:** Cloudflare Enterprise Edge (Full Strict SSL / WAF / CDN)

1. High-Level Production Architecture & Traffic Flow

This production setup follows an enterprise **Defense-in-Depth** strategy. The actual Hostinger KVM 4 server IP is completely hidden behind Cloudflare's Edge Anycast network, safeguarding your infrastructure against DDoS attacks, automated exploit scanners, and direct malicious traffic.

1. Visitor Request	End-user accesses https://cortexissolutionhub.com via browser or mobile.
2. Cloudflare Edge (DDoS, WAF, CDN, SSL)	• **DNS & Anycast:** Lightning-fast global resolution. • **WAF & Bot Fight:** Blocks malicious bots, SQL injection, and XSS attacks. • **CDN Edge Caching:** Static assets (CSS, JS chunks, WebP images, 177 prerendered HTML routes) served from local edge nodes in Karachi, Lahore, Dubai, etc. • **Edge SSL:** Terminates client HTTPS with valid public SSL cert.
3. KVM 4 Firewall (UFW) (Origin Cloaking)	• Ports 80 & 443 strictly accept connections **ONLY from Cloudflare's official IP ranges**. • Direct requests to the raw server IP are dropped immediately by the kernel firewall.
4. Nginx Reverse Proxy (SSL Termination & Headers)	• Verifies the 15-year **Cloudflare Origin CA Certificate**. • Restores real client IP addresses via CF-Connecting-IP header. • Enables HTTP/2, Gzip/Brotli compression, and forwards traffic to Docker port 5000.
5. Docker Compose Network (Isolated Backend Services)	• cortexis-website: Express API + built React bundle on port 5000. • cortexis-postgres: PostgreSQL 16 on private network (port 5432 not exposed). • cortexis-redis: Redis 7 memory cache & rate limiter (port 6379 not exposed). • gotcha-rush & typedash: Embedded interactive game engines.

2. Phase 1: Cloudflare Setup (CDN, WAF & SSL)

Step 1.1: Nameserver Delegation

Log into your domain registrar (Hostinger Domain Management), navigate to **DNS / Nameservers**, change from Hostinger Default Nameservers to Custom, and input Cloudflare's assigned nameservers (e.g. ns1.cloudflare.com, ns2.cloudflare.com).

Step 1.2: DNS Records Configuration

Inside the Cloudflare DNS dashboard, create two essential A records pointing to your Hostinger KVM 4 IP address with the **Orange Cloud (Proxied)** status enabled:

Type	Name	IPv4 Target	Proxy Status	TTL
A	@ (apex)	<YOUR_KVM4_IP_ADDRESS>	Proxied (Orange Cloud)	Auto
A	www	<YOUR_KVM4_IP_ADDRESS>	Proxied (Orange Cloud)	Auto

Step 1.3: SSL/TLS Encryption Mode

In Cloudflare navigate to **SSL/TLS > Overview**. Select **Full (Strict)**. This guarantees end-to-end encryption between the visitor, Cloudflare Edge, and your Nginx server on KVM 4.

Step 1.4: Generate Cloudflare Origin CA Certificate (15-Year Validity)

Navigate to **SSL/TLS > Origin Server** and click **Create Certificate**. Leave defaults (RSA 2048, hostnames cortexissolutionhub.com and *.cortexissolutionhub.com, 15 years validity). Copy and save the generated **Origin Certificate** and **Private Key** into secure text files; we will install them onto Nginx in Phase 3.

Step 1.5: Security & WAF Configuration

- **Security > Bots:** Turn on **Bot Fight Mode** (challenges known scrapers and automated DDoS bots).
- **Security > WAF > Rate Limiting:** Set up a rule for /api/auth/* (e.g. max 10 requests per minute per IP) to prevent credential stuffing.

Step 1.6: CDN Caching Optimization

Navigate to **Caching > Cache Rules** and create a rule:

- If URI Path starts with /static/ or ends with .webp, .png, .jpg, .js, .css → **Cache Eligibility: Eligible for Cache**, Edge TTL: 1 month. This ensures KVM 4 bandwidth is never consumed by repeated asset requests.

3. Phase 2: Hostinger KVM 4 Server Provisioning & Hardening

Connect to your Hostinger KVM 4 server via SSH using your preferred terminal (PuTTY or PowerShell):
ssh root@<YOUR_KVM4_IP_ADDRESS>

Step 2.1: System Updates & Baseline Dependencies

```
# 1. Update package indices and upgrade existing packages
sudo apt update && sudo apt upgrade -y

# 2. Install essential system tools and network utilities
sudo apt install -y curl wget git ufw htop fail2ban unzip software-properties-common ca-certificates
```

Step 2.2: UFW Firewall Hardening (Locking Ports to Cloudflare Only)

Critical Security Rule: To prevent attackers from bypassing Cloudflare and hammering your server directly via its raw IP address, we configure UFW to allow Web Traffic (Ports 80 & 443) *exclusively* from Cloudflare's official IP ranges:

```
# 1. Allow SSH connection first to avoid getting locked out!
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp

# 2. Whitelist official Cloudflare IPv4 ranges for HTTP (80) and HTTPS (443)
for ip in 173.245.48.0/20 103.21.244.0/22 103.22.200.0/22 103.31.4.0/22 \
141.101.64.0/18 108.162.192.0/18 190.93.240.0/20 188.114.96.0/20 \
197.234.240.0/22 198.41.128.0/17 162.158.0.0/15 104.16.0.0/13 \
104.24.0.0/14 172.64.0.0/13 131.0.72.0/22; do
sudo ufw allow proto tcp from $ip to any port 80,443
done

# 3. Enable the firewall
sudo ufw enable && sudo ufw status
```

Step 2.3: Install Docker Engine & Docker Compose

```
# 1. Add official Docker GPG key and repository
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] \
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# 2. Install Docker and Compose plugin
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

# 3. Verify Docker status
docker --version && docker compose version
```

Step 2.4: Install Nginx Web Server

```
sudo apt install -y nginx
sudo systemctl enable nginx && sudo systemctl start nginx
```

4. Phase 3: Nginx Reverse Proxy & Cloudflare Origin SSL

Nginx serves as our frontline reverse proxy on KVM 4. It terminates the Cloudflare Origin SSL, restores the genuine user IP address from CF-Connecting-IP, and proxies incoming requests to our Docker container running on 127.0.0.1:5000.

Step 3.1: Save Cloudflare Origin Certificate and Private Key

```
# 1. Create certificate directory
sudo mkdir -p /etc/ssl/cortexis

# 2. Create the certificate file and paste the Certificate from Cloudflare Step 1.4
sudo nano /etc/ssl/cortexis/origin.pem

# 3. Create the private key file and paste the Private Key from Cloudflare Step 1.4
sudo nano /etc/ssl/cortexis/origin.key

# 4. Secure key permissions
sudo chmod 600 /etc/ssl/cortexis/origin.key
sudo chmod 644 /etc/ssl/cortexis/origin.pem
```

Step 3.2: Production Nginx Server Block Configuration

Create the site configuration file: sudo nano /etc/nginx/sites-available/cortexis and paste the following tested configuration:

```
# Define Cloudflare IP ranges to properly restore real visitor IPs
set_real_ip_from 173.245.48.0/20;
set_real_ip_from 103.21.244.0/22;
set_real_ip_from 103.22.200.0/22;
set_real_ip_from 103.31.4.0/22;
set_real_ip_from 141.101.64.0/18;
set_real_ip_from 108.162.192.0/18;
set_real_ip_from 190.93.240.0/20;
set_real_ip_from 188.114.96.0/20;
set_real_ip_from 197.234.240.0/22;
set_real_ip_from 198.41.128.0/17;
set_real_ip_from 162.158.0.0/15;
set_real_ip_from 104.16.0.0/13;
set_real_ip_from 104.24.0.0/14;
set_real_ip_from 172.64.0.0/13;
set_real_ip_from 131.0.72.0/22;
real_ip_header CF-Connecting-IP;

# 1. Redirect HTTP to HTTPS
server {
listen 80;
server_name cortexissolutionhub.com www.cortexissolutionhub.com;
return 301 https://$host$request_uri;
}

# 2. HTTPS Server Block
server {
listen 443 ssl http2;
server_name cortexissolutionhub.com www.cortexissolutionhub.com;

ssl_certificate /etc/ssl/cortexis/origin.pem;
ssl_certificate_key /etc/ssl/cortexis/origin.key;
ssl_protocols TLSv1.2 TLSv1.3;
ssl_ciphers HIGH:!aNULL:!MD5;
ssl_prefer_server_ciphers on;

client_max_body_size 50M;

# Proxy all traffic to Dockerized Node.js API + Frontend bundle
location / {
proxy_pass http://127.0.0.1:5000;
proxy_http_version 1.1;
proxy_set_header Upgrade $http_upgrade;
proxy_set_header Connection 'upgrade';
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto https;
proxy_cache_bypass $http_upgrade;
proxy_read_timeout 90s;
}
}
```

Step 3.3: Test and Activate Nginx Configuration

```
sudo ln -s /etc/nginx/sites-available/cortexis /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t && sudo systemctl reload nginx
```

5. Phase 4: Docker Application Deployment

Step 4.1: Clone the GitHub Repository

```
sudo mkdir -p /var/www
cd /var/www
sudo git clone https://github.com/abdullah5654/cortexis-website.git cortexis
cd cortexis
```

Step 4.2: Configure Production Environment Variables (.env)

Create the root .env file: nano /var/www/cortexis/.env with your production secrets:

```
PORT=5000
NODE_ENV=production
POSTGRES_PASSWORD=your_super_strong_postgres_password_2026
JWT_SECRET=your_ultra_secure_jwt_secret_64_characters_random_string
REDIS_PASSWORD=your_super_strong_redis_password_2026
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=contact@cortexissolutionhub.com
SMTP_PASS=your_smtp_app_password
CONTACT_TO=contact@cortexissolutionhub.com
ADMIN_ALERT_TO=admin@cortexissolutionhub.com
```

Step 4.3: Launch Docker Containers

```
# Build images and launch all microservices in detached mode
docker compose up -d --build

# Verify running containers
docker compose ps
```

Step 4.4: Healthcheck and Telemetry Verification

```
# 1. Test local API health endpoint
curl -I http://127.0.0.1:5000/health

# 2. Inspect live application container logs
docker compose logs -f cortexis-website
```

6. Phase 5: Automated Backups & 1-Command Updates

Automated Nightly PostgreSQL Backup Cron Job:

Add a cron job to automatically backup the database every night at 2:00 AM and retain the last 7 days of backups:

```
# Edit crontab
sudo crontab -e

# Add this nightly backup line:
0 2 * * * docker exec cortexis-postgres pg_dump -U cortexis cortexis | gzip > /var/backups/cortexis_$(date
+\%F).sql.gz && find /var/backups/ -name "cortexis_*.sql.gz" -mtime +7 -delete
```

Deploying Future Updates (1-Command Update Script):

```
cd /var/www/cortexis
git pull origin main
docker compose up -d --build
docker image prune -f
```

7. Pre-Launch Verification Checklist

Component	Verification Test	Status
Domain & DNS	Cloudflare nameservers active; A records proxied (Orange Cloud ON).	**READY**
SSL Encryption	SSL mode set to Full (Strict); Origin CA cert installed in Nginx.	**READY**
WAF & DDoS	Bot Fight Mode active; Port 80/443 locked to Cloudflare IPs via UFW.	**READY**
Real IP Logging	Nginx restores real visitor IP from CF-Connecting-IP header.	**READY**
CDN Edge Caching	Static bundle, courses, tools, and images cached on edge.	**READY**
PostgreSQL 16	Millions-scale table claude_pro_applications and indices created.	**READY**
Redis 7 Caching	Course catalogue and rate limiters operational with automatic DB fallback.	**READY**
Claude Pro Incentive	Hero notification, announcement bar, study-time tracker & auto-apply active.	**READY**
Nightly Backups	PostgreSQL automated pg_dump gzip cron job scheduled.	**READY**

Architectural Guarantee: By combining Cloudflare's Edge Anycast Network with Hostinger KVM 4's generous 16 GB RAM and 4 vCPU allocation, Cortexis Solution Hub achieves high availability, multi-tier DDoS resistance, sub-millisecond Redis response caching, and frictionless horizontal scaling with 0 database bottlenecks.